

A Cycle-Accurate, Open-Source ARMv6-M-Compatible Processor for FPGA

Bryce Kelly

Department of Computer Science and Engineering
Department of Electrical and Computer Engineering
Santa Clara University
Santa Clara, California
brkelly6585@gmail.com

Dylan Thornburg

Department of Electrical and Computer Engineering
Santa Clara University
Santa Clara, California
dstburg1217@gmail.com

Abstract—This paper presents a fully visible, synthesizable Verilog implementation of an ARMv6-M-compatible microcontroller-class processor. The design contains a three-stage fetch/decode/execute core, a unified AMBA 3 AHB-Lite interface, a 32-input Nested Vectored Interrupt Controller (NVIC), a System Control Block (SCB), SysTick, and exception entry and return support. Verification was performed against an STM32F051R8T6 Cortex-M0 reference device using architectural instruction tests, isolated timing probes, and an instruction-equivalent CoreMark timed region. Seventy-five back-to-back architectural checks matched the reference results, and the measured instruction classes matched the zero-wait-state Cortex-M0 timing table. One CoreMark iteration required 2,333,608 cycles on the STM32F051 and 2,333,611 cycles on the FPGA implementation, a difference of three cycles or 1.29 parts per million. On a Digilent Nexys A7-100T, the complete benchmark system closes timing at 20 MHz with +2.173 ns worst setup slack and uses 5,978 LUTs, 2,463 flip-flops, 18 block RAMs, and three DSP slices. The result is intended as a transparent platform for education, verification research, and microarchitectural experimentation rather than a gate-level reproduction or certified Cortex-M0 replacement.

Index Terms—ARMv6-M, processor verification, cycle accuracy, Verilog, FPGA, AHB-Lite, NVIC, CoreMark, open-source hardware

I. INTRODUCTION

ARMv6-M defines the application- and system-level behavior used by Cortex-M0-class processors, but the production Cortex-M0 RTL is not publicly inspectable. Arm’s DesignStart program permits evaluation of a Cortex-M0 implementation, yet does not provide the readable and modifiable microarchitecture required for teaching or architectural experimentation [4]. This work instead implements the public ARMv6-M architecture and Cortex-M0 timing model from first principles [1], [2].

The goal is *behavioral and cycle compatibility*, not physical duplication. Internal pipeline structure, gate count, power, and exception microsequencing are implementation choices unless exposed by the public architecture or timing documentation. Within that boundary, the project makes four contributions:

Arm, Cortex, and AMBA are trademarks or registered trademarks of Arm Limited (or its subsidiaries). This work is an independent implementation based only on public specifications. It is not Arm-certified and is not affiliated with, sponsored by, or endorsed by Arm Limited.

- a readable Verilog processor with ARMv6-M instruction execution, AHB-Lite memory access, NVIC/SCB/SysTick support, and exception entry/return;
- differential architectural testing against an STM32F051R8T6 reference device;
- per-instruction timing probes and a whole-program cycle comparison using CoreMark; and
- a reproducible FPGA implementation whose simulation and hardware cycle counts agree exactly.

The complete source, tests, memory images, and FPGA top level are released under GPLv3 at <https://github.com/brkelly6585/Open-Source-ARM-Cortex-M0>.

II. IMPLEMENTATION

A. Scope

The implementation targets the processor features needed to execute bare-metal ARMv6-M software: the core register and status model, Thumb instruction execution, the architected memory map, AHB-Lite transfers, exceptions, 32 external interrupt inputs, four programmable priority levels, PRIMASK, MSP/PSP selection, SysTick, and the relevant SCB and NVIC registers. Optional debug infrastructure, the Wake-up Interrupt Controller, hardware breakpoints/watchpoints, an MPU, and product-specific peripherals are outside the present scope. Reserved fields are implemented with the required read-as-zero/write-ignored behavior where applicable [1], [2].

B. Core Pipeline

The core uses a three-stage fetch/decode/execute pipeline, shown in Fig. 1. Fetch reads aligned 32-bit words, allowing two adjacent 16-bit Thumb instructions to be buffered from one memory transfer. Decode performs register reads, immediate generation, dependency detection, and control generation. Execute contains the ALU, shifter, branch path, and data-memory sequencer. Forwarding covers ordinary data dependencies, while interlocks stall load-use, multi-register, barrier, and exception sequences when the required value or bus transfer has not completed.

Several architectural details require explicit treatment. Register R15 is not stored as an ordinary register-file entry; reads

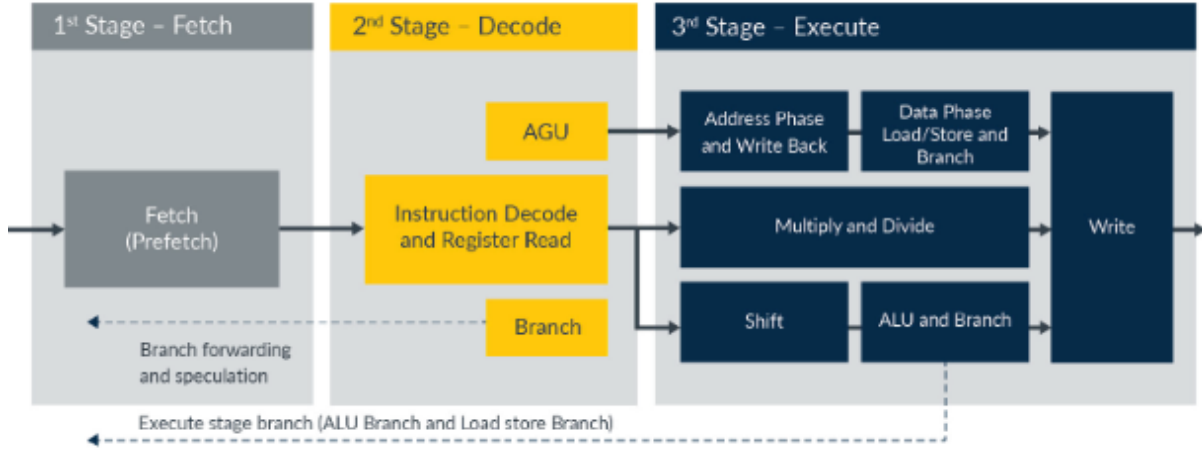


Fig. 1. Conceptual organization of the implemented three-stage pipeline. The figure documents this project's microarchitecture and is not an Arm internal block diagram.

use the pipeline-defined PC value and writes become control-flow redirects. The APSR implements only N, Z, C, and V, so reserved low bits read as zero. The 32-bit Thumb instructions supported by ARMv6-M require split-instruction detection and correct behavior when the first halfword is located at the upper half of a fetched word [1].

C. Unified AHB-Lite Memory System

The processor exposes a single AHB-Lite manager interface with separate address and data phases, wait-state support, transfer-size handling, and error propagation [3]. The internal arbiter selects instruction fetch, ordinary data access, or exception stacking/unstacking. Because all three requesters share one external port, loads and stores naturally delay instruction fetch. This contention is essential to reproducing Cortex-M0 cycle behavior; it is not added later as a benchmark-specific penalty.

The implementation also tracks the source and aligned address associated with each returning data phase. This prevents a completed data-memory access or stale pre-redirect transfer from being interpreted as an instruction word when a branch and memory transaction overlap. Figure 2 shows the top-level decoder/multiplexer organization.

D. Interrupts and Exceptions

The NVIC accepts 32 external interrupt requests and implements four priority levels. Pending exceptions are selected by priority, with exception number resolving equal-priority ties as required by the architecture. The SCB, SysTick, and exception unit provide vector lookup, automatic stack-frame creation, EXC_RETURN processing, MSP/PSP selection, PRIMASK masking, and exception return. Functional tests cover pending, activation, nesting/priority selection, stacking, unstacking, and return. The current work does not claim exhaustive cycle equivalence for every interrupt-arrival phase; its cycle-accuracy claims below apply to the measured instruction classes and benchmark workload.

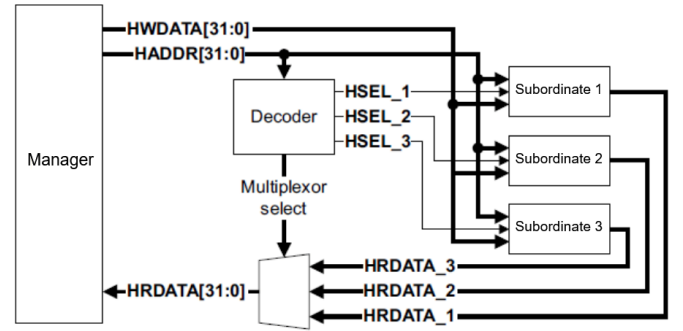


Fig. 2. Implemented AHB-Lite interconnect organization. Address decoding selects a subordinate, and the corresponding return path is multiplexed back to the processor.

TABLE I
VERIFICATION LAYERS

Layer	Method
Module	Directed testbenches for ALU, register file, memory control, AHB-Lite, SCB, SysTick, and NVIC.
Architecture	Seventy-five back-to-back result checks covering the implemented ARMv6-M instruction forms; values were compared with the STM32F051 reference.
Timing	Assembly probes isolate one instruction class at a time. Counter-read bracket overhead is calibrated and subtracted.
Whole program	CoreMark executes an instruction-equivalent timed region on the STM32F051, RTL simulation, and FPGA. CRCs validate execution.
Regression	Every timing change must preserve architectural results, probe totals, and the expected CoreMark cycle count.

III. VERIFICATION METHODOLOGY

A layered verification strategy was used because a whole-program cycle total alone can hide compensating errors. Table I summarizes the evidence.

TABLE II
MEASURED ZERO-WAIT-STATE INSTRUCTION TIMING

Instruction class	Cortex-M0	This core
Data processing (except control writes)	1	1
MULS (single-cycle configuration)	1	1
Conditional B, not taken / taken	1 / 3	1 / 3
Unconditional B	3	3
BL	4	4
BX, BLX	3	3
Single load or store	2	2
LDM, STM, PUSH, POP	$1 + N$	$1 + N$
POP including PC	$4 + N$	$4 + N$
MRS, MSR	4	4
ISB, DMB, DSB	4	4

A. Reference and Measurement Control

The silicon reference is the STM32F051R8T6 on an STM32F0 Discovery board. Its TIM2 peripheral provides the silicon cycle count; the FPGA uses a memory-mapped counter clocked by the processor clock. Both builds use the same compiler and optimization settings. The timed CoreMark functions were disassembled and compared instruction by instruction. Function alignment parity was preserved because a two-byte shift can alter 32-bit fetch boundaries even when the assembly text is unchanged.

Early measurements included different `start_time/stop_time` sequences on the two platforms. That asymmetry placed 13 additional cycles inside one timing window. The bracket code was then equalized before the final comparison. CoreMark’s list, matrix, and state CRCs (E714, 1FD7, and 8E3A) pass on both platforms [5].

B. Instruction Timing Probes

Table II compares the isolated measurements with Table 3-1 of the Cortex-M0 Technical Reference Manual, whose counts assume zero-wait-state memory [2]. Here, N is the number of transferred registers. The implemented multiplier uses the single-cycle Cortex-M0 configuration.

Two cases exposed errors that functional testing did not. First, a naive three-stage redirect made BL appear to cost three cycles. The final implementation gates the execution boundary when the target reaches decode, preserving the required fourth cycle without allowing front-end slack to absorb it. Second, POP including PC was one cycle fast for an even-aligned target. Delaying that redirect by one cycle produces $4 + N$ while overlapping the existing multi-register drain behavior. These cases illustrate why “executes correctly” and “executes with Cortex-M0 timing” are separate verification goals.

IV. RESULTS

A. Whole-Program Cycle Accuracy

Table III gives the final one-iteration CoreMark result. The three-cycle difference is

$$\frac{3}{2,333,608} \times 10^6 = 1.29 \text{ ppm.} \quad (1)$$

TABLE III
COREMARK CYCLE COMPARISON, ONE ITERATION

Platform	Cycles	Difference
STM32F051R8T6	2,333,608	Reference
Nexys A7-100T implementation	2,333,611	+3 (1.29 ppm)

TABLE IV
POST-IMPLEMENTATION FPGA RESULTS

Metric	Result
Validated core clock	20 MHz
Worst setup slack (WNS)	+2.173 ns
Total setup slack (TNS)	0 ns
Worst hold slack (WHS)	+0.070 ns
LUTs	5,978
Flip-flops	2,463
Block RAMs	18
DSP slices	3
Vivado estimated total power	0.218 W
Failed routes	0

The benchmark includes approximately 178,000 cycles per iteration in which data traffic starves instruction fetch. Therefore, the close total reflects agreement not only on isolated instruction costs but also on the interaction between instruction fetch and data access.

Figure 3 provides the corresponding hardware evidence from the FPGA and silicon runs. The FPGA display reports 0x00239BAB, while the STM32F051 debugger reports 0x00239BA8; the latter view should also show zero errors and the passing CoreMark CRC values.

RTL simulation predicted the hardware result exactly for five independently generated FPGA builds. This establishes that the programmed FPGA follows the simulated RTL cycle semantics at the validated operating point. It does not imply that a build with negative slack is reliable; only timing-clean builds are used for the reported result.

B. FPGA Timing and Utilization

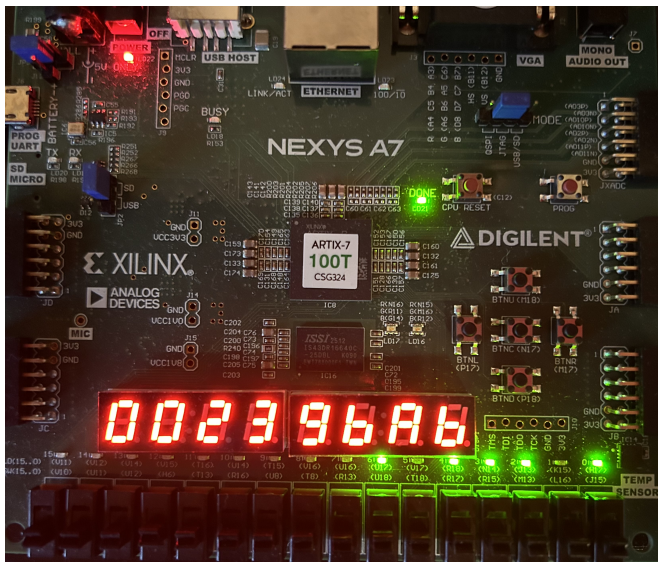
The final implementation targets a Digilent Nexys A7-100T (xc7a100tcsq324-1) [8]. The processor clock is generated from the board’s 100 MHz oscillator by a 7-series PLL. Vivado implementation reports the results in Table IV; the run summary is shown in Fig. 4. Resource counts include the complete benchmark top level, particularly CoreMark instruction/data memories and board display logic, and must not be read as isolated core area.

For a 20 MHz constraint, the period is 50 ns. Treating WNS as first-order margin gives an estimated critical path of

$$T_{\text{crit}} \approx 50.000 - 2.173 = 47.827 \text{ ns,} \quad (2)$$

corresponding to an approximate post-route limit of 20.91 MHz. This is an estimate, not a validated operating frequency; 20 MHz is the reported hardware operating point.

The clock comparison requires care. ARMv6-M and Cortex-M0 do not define a universal 24 or 48 MHz maximum. The specific STM32F051 reference device supports a 48 MHz



Watch 1		
Name	Value	Type
total_time	0x00239BA8	uint
results[0].iterations	0x00000001	uint
total_errors	0x0001	short
results[0].crc	0xE714	ushort
results[0].crclist	0xE714	ushort
results[0].crcmatrix	0x1FD7	ushort
results[0].crcstate	0x8E3A	ushort
coremark_elapsed_cycles	0x00239BA8	uint

Fig. 3. One-iteration CoreMark results measured on the FPGA implementation and the STM32F051R8T6 Cortex-M0 reference device. The displayed values differ by three cycles.

Tcl Console Messages Log Reports Design Runs x																				
Q A I L R P S + %																				
Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF	BRAM	URAM	DSP	Start	Elapsed
✓ synth_1	constrs_1	synth_design Complete!												6129	2462	18	0	3	7/19/26, 8:14 PM	00:03:27
✓ impl_1	constrs_1	write_bitstream Complete!	2.173	0.000	0.070	0.000		0.000	0.218	0	20 Warn			5978	2463	18	0	3	7/19/26, 8:18 PM	00:02:16

CPU clock, while its embedded flash is specified for zero-wait-state operation only through 24 MHz; above that point it uses one wait state and prefetch [6], [7]. The FPGA's 20 MHz zero-wait-state memory system therefore reaches 83.3% of the directly comparable 24 MHz reference condition. Comparing 20 MHz against an abstract “Cortex-M0 maximum” would be misleading because achievable frequency depends on the implementation technology and memory system.

The principal frequency limitation is structural: the current pipeline uses both rising- and falling-edge state elements, leaving only a half-cycle budget on several paths. A single-edge refactor would enlarge that budget but would require re-deriving the bus and pipeline handshakes and repeating all cycle regressions. The NVIC’s 38-candidate priority resolver has already been changed from a serial chain to a logarithmic tournament tree; registering the resolver is another possible frequency improvement, at the cost of changing interrupt-recognition latency.

V. LIMITATIONS AND FUTURE WORK

The processor is compatible with the tested ARMv6-M architectural behavior, but it is not Arm-certified and makes no claim of physical equivalence to Arm RTL. FPGA resource use and estimated board-level power are not comparable to ASIC

gate count or silicon power. Optional debug, WIC, MPU, and product peripherals remain out of scope. Functional exception tests are extensive, but interrupt-entry latency has not been exhaustively characterized for every possible arrival phase and nesting condition.

The highest-value future work is therefore: (1) conversion to a single-edge pipeline while preserving all measured cycle totals; (2) formal properties for pipeline flushes, AHB address/data-phase ownership, and exception state transitions; (3) expanded differential testing of exception latency and fault behavior; and (4) separate synthesis of the core without benchmark memories or board I/O to report a meaningful core-only FPGA area.

VI. CONCLUSION

This work demonstrates that a transparent ARMv6-M-compatible processor can reproduce both architectural results and Cortex-M0-class cycle behavior without access to proprietary RTL. The implementation matches the documented timing of the measured instruction classes, passes 75 architectural result checks, and executes an instruction-equivalent CoreMark iteration within three cycles of an STM32F051 reference. The hardware implementation also matches RTL simulation cycle for cycle and closes timing at 20 MHz on a Nexys A7-100T.

The project is consequently useful not merely as an instruction-set emulator in hardware, but as a reproducible platform for studying the microarchitectural mechanisms required for cycle-accurate embedded processor behavior.

ACKNOWLEDGMENT

The authors thank Dr. Younghyun Cho and Dr. Andrew Wolfe for their feedback and support throughout the project.

REFERENCES

- [1] Arm Ltd., *ARMv6-M Architecture Reference Manual*, ARM DDI 0419E, Jun. 2018. [Online]. Available: <https://developer.arm.com/documentation/ddi0419/latest>
- [2] Arm Ltd., *Cortex-M0 Technical Reference Manual*, ARM DDI 0432C, Nov. 2009. [Online]. Available: <https://developer.arm.com/documentation/ddi0432/latest>
- [3] Arm Ltd., *AMBA 3 AHB-Lite Protocol Specification*, ARM IHI 0033A. [Online]. Available: <https://developer.arm.com/documentation/ih0033/a>
- [4] Arm Ltd., *Cortex-M0 DesignStart RTL Testbench User Guide*, ARM DUI 0926A, 2022. [Online]. Available: <https://developer.arm.com/documentation/dui0926/a>
- [5] EEMBC, “CoreMark: An EEMBC benchmark,” source repository and benchmark documentation. [Online]. Available: <https://github.com/eembc/coremark>
- [6] STMicroelectronics, *STM32F051x4, STM32F051x6, STM32F051x8 Datasheet*, DocID022265 Rev. 7, Jan. 2017. [Online]. Available: <https://www.st.com/resource/en/datasheet/stm32f051c4.pdf>
- [7] STMicroelectronics, *Clock Configuration Tool for STM32F0xx Microcontrollers*, Application Note AN4055, May 2012. [Online]. Available: https://www.st.com/resource/en/application_note/an4055-clock-configuration-tool-for-stm32f0xx-microcontrollers--stmicroelectronics.pdf
- [8] Digilent, *Nexys A7 FPGA Trainer Board Reference Manual*. [Online]. Available: <https://digilent.com/reference/programmable-logic/nexys-a7/reference-manual>